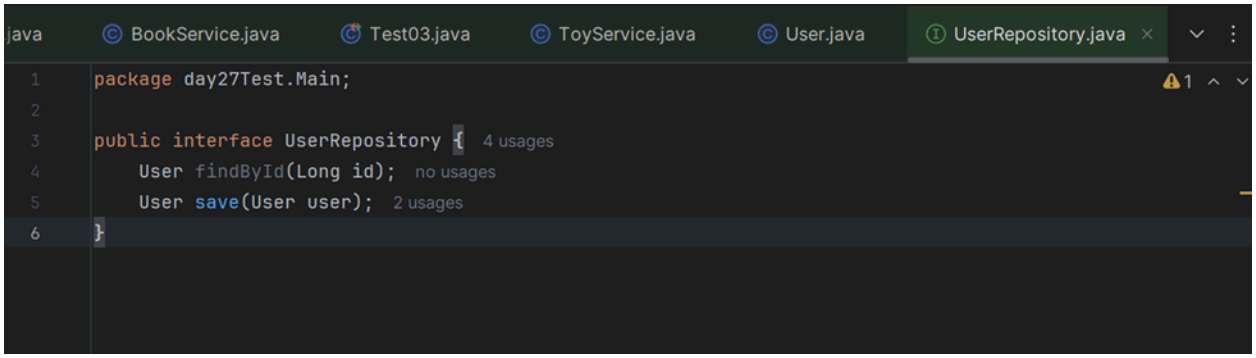
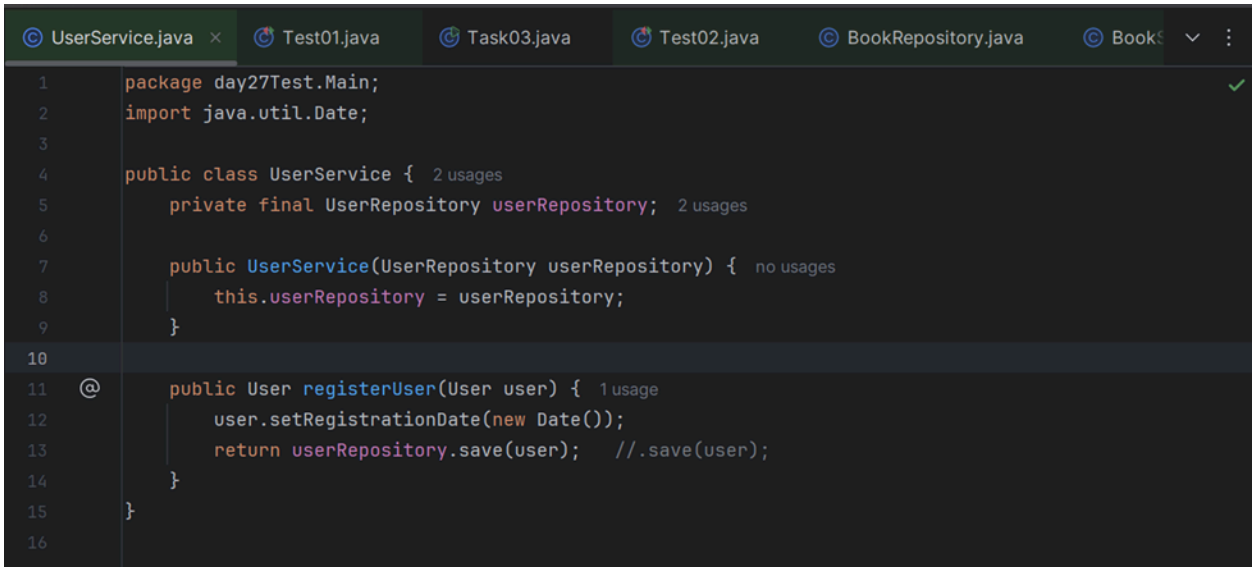
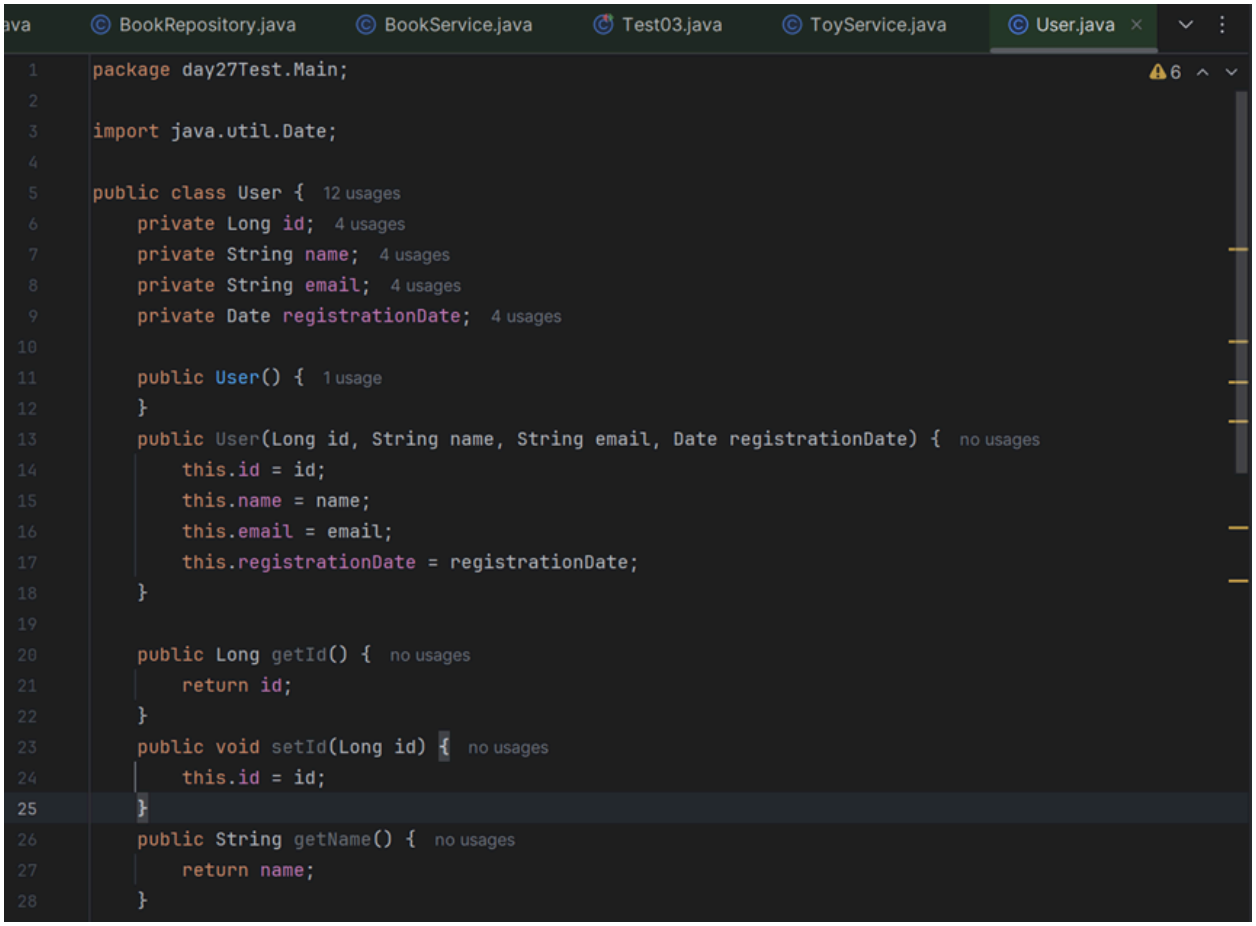

java
BookService.java
Test03.java
ToyService.java
User.java
UserRepository.java
package day27Test.Main;
public interface UserRepository { 4 usages
User findById(Long id); no usages
User save(User user); 2 usages
}


UserService.java
Test01.java
Task03.java
Test02.java
BookRepository.java
package day27Test.Main;
import java.util.Date;
public class UserService { 2 usages
private final UserRepository userRepository; 2 usages
public UserService(UserRepository userRepository) { no usages
this.userRepository = userRepository;
}
public User registerUser(User user) { 1 usage
user.setRegistrationDate(new Date());
return userRepository.save(user); //.save(user);
}
}


ava
BookRepository.java
BookService.java
Test03.java
ToyService.java
User.java
6
package day27Test.Main;
import java.util.Date;
public class User { 12 usages
private Long id; 4 usages
private String name; 4 usages
private String email; 4 usages
private Date registrationDate; 4 usages
public User() { 1 usage
}
public User(Long id, String name, String email, Date registrationDate) { no usages
this.id = id;
this.name = name;
this.email = email;
this.registrationDate = registrationDate;
}
public Long getId() { no usages
return id;
}
public void setId(Long id) { no usages
this.id = id;
}
public String getName() { no usages
return name;
}

```java
public class Test01 {

    @InjectMocks  1 usage
    private UserService userService;


    @Test
    void testRegisterUser() {


        User newUser = new User();
        newUser.setName("Himanshu Rajpoot");


        when(userRepository.save(any(User.class))).thenReturn(newUser);


        User registeredUser = userService.registerUser(newUser);

        assertNotNull(registeredUser);
        assertNotNull(registeredUser.getRegistrationDate());
    }
}
```

Test02.java | BookRepository.java | BookService.java | Test03.java | ToyService.java

```java
package day27Test.Main;

public class ToyService {  2 usages

    public String getToyName(int id) {  3 usages
        if (id == 1) {
            return "Lego";
        } else {
            return getFallbackName();
        }
    }

    public String getFallbackName() {  3 usages
        return "Unknown Toy";
    }
}
```

Test01.java | Task03.java | Test02.java | BookRepository.java | BookService.java | Test0

```java
@ExtendWith(MockitoExtension.class)
class Test02 {

    @Spy  5 usages
    ToyService spyToyService;

    @Test
    void method1() {

        String name = spyToyService.getToyName( id: 1);

        assertEquals( expected: "Lego", name);

        verify(spyToyService, times( wantedNumberOfInvocations: 1)).getToyName( id: 1);
    }

    @Test
    void method2() {
        doReturn( toBeReturned: "Default toy").when(spyToyService).getFallbackName();

        String name = spyToyService.getToyName( id: 3);


        assertEquals( expected: "Default toy", name);


        verify(spyToyService, times( wantedNumberOfInvocations: 1)).getFallbackName();
    }
}
```

Test02.java | BookRepository.java | BookService.java | Test03.java | ToyService.java

```java
package day27Test.Main;

public class BookRepository {  no usages
    public int getBookCount() {  no usages
        System.out.println("assume calling real getBookCount() from the database.");
        return 10;
    }
}
```

Test02.java | BookRepository.java | BookService.java | Test03.java | ToyService.java

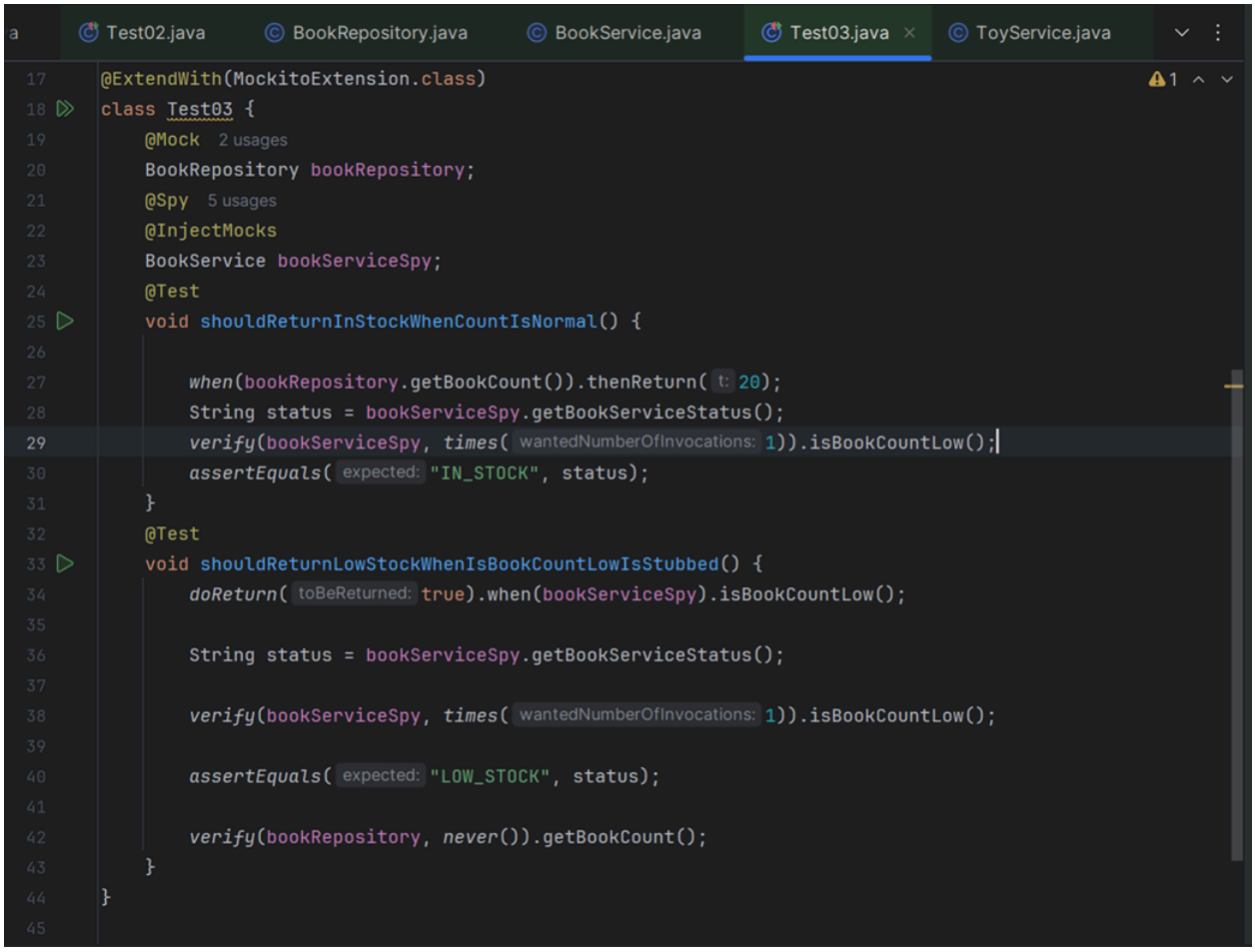
```java
package day27Test.Main;

public class BookService {  no usages
    private final BookRepository bookRepository;  2 usages


    public BookService(BookRepository bookRepository) {  no usages
        this.bookRepository = bookRepository;
    }


    public boolean isBookCountLow() {  1 usage
        int count = bookRepository.getBookCount();
        System.out.println("Real method isBookCountLow() called.");
        return count < 10;
    }


    public String getBookServiceStatus() {  no usages
        if (isBookCountLow()) {
            return "LOW_STOCK";
        }
        return "IN_STOCK";
    }
}
```



Task 09:

In the context of Automation Testing, how does the Page Object Model (POM) improve test maintainability?

1. POM creates separate XML files for each UI element which reduces hardcoding in test scripts.

2. POM introduces a single test method for all page elements, reducing redundancy.

3. POM stores test data in property files which helps in data-driven testing.

4. POM abstracts UI elements as objects in separate classes, which isolates changes in the UI from the test logic.

Task 10:

In a Page Object Model (POM) framework, where should test data ideally reside?

1. In the page class itself for better cohesion.

2. In the test script, embedded as literals for simplicity.

3. In external files such as JSON, Excel, or property files to separate logic from data.

4. In browser cookies to simulate user sessions.

Task 11:

A team is building a Page Object Model (POM) automation framework. However, they observe test failures due to frequent changes in UI locators across environments. What should they focus on improving in their automation approach?

1. Use absolute XPath locators as they are more stable across environments.

2. Store locators in external property files and abstract them within page classes to isolate UI changes.

3. Implement a retry mechanism in every step to counter locator issues.

4. all locators to dynamic XPath expressions at runtime.

Task 12:

Why are mock objects commonly used in JUnit tests for services that call external APIs?

1. They simulate external APIs and allow testing of service logic in isolation without depending on actual API availability or response time.

2. They provide GUI forms for mocking user input.

3. They automatically generate alternate test scenarios.

4. They allow browser-based execution for all test cases.

Task 13:

You've written a JUnit test for an API that sends email notifications. Running the test actually sends emails, causing issues. How should this be addressed?

1. Replace the test logic with a mock that simulates email service behavior.

2. Remove the test as it affects production systems.

3. Use assertNull() on email object to prevent sending.

4. Allow emails but send them to a dummy address.

Task 14:

Which of the following best describes the role of test suites in JUnit?

1. Test suites are used to group test methods within a single class for reporting.

2. Test suites allow grouping multiple test classes and running them together, often used for regression or integration testing.

3. Test suites run a single test class multiple times with different parameters.

4. Test suites are used to auto-generate mock objects for integration with third-party services.

Task 15:

Why are mock objects used in JUnit testing, especially in enterprise applications?

1. They eliminate the need for assertions by capturing logs during execution.

2. They replace real dependencies like databases or web services to test components in isolation and ensure reliability.

3. They automate UI validations by replicating the behavior of actual user interactions.

4. They allow for runtime generation of test data based on machine learning techniques.

Task 16:

What does the annotation @Disabled do in a JUnit test?

1. It is used to generate random input values for the test cases.

2. It ensures that the test case will only run if another test fails.

3. It replaces the test method with a mock version automatically during runtime.

4. It marks a test case to be skipped during execution without deleting or commenting out the code.

Task 17:

What is the primary purpose of using assertions in JUnit tests?

1. Assertions are used to execute code blocks before and after the test cases for logging purposes.

2. Assertions ensure that certain exceptions are thrown during the execution of test methods.

3. Assertions define expected outcomes in tests and help automatically verify the correctness of code behavior.

4. Assertions are used to configure the order of test execution in test suites.

Task 18:

What is one of the key differences between manual testing and automation testing from a scalability perspective?

1. Manual testing scales better because it allows human intuition in exploratory testing scenarios.

2. Automation testing is ideal for scaling regression tests across builds and environments as scripts can run unattended.

3. Manual testing uses more reliable tools compared to automation which may fail silently.

4. Automation testing allows for ad-hoc testing with high flexibility in test case creation.

Task 19:

What is the purpose of the assertThat() method in conjunction with the Hamcrest library in JUnit?

1. It improves performance by running multiple assertions in parallel threads.

2. It allows developers to create GUI-based test results with better formatting.

3. It provides a more flexible and readable way to define expectations using matchers like containsString, hasItems, etc.

4. It is primarily used to automate exception handling by wrapping assertions in try-catch blocks.

Task 20:

How does the concept of timeout help improve the reliability of test execution in JUnit?

1. Timeout ensures that tests always return true after a specified duration, indicating success.

2. Timeout ignores any assertion failures if a method takes too long to execute.

3. Timeout helps identify tests that hang or take unusually long to execute, allowing early detection of performance issues.

4.Timeout marks the test as passed even if it is interrupted by a system signal or thread abort.